CS 423
Operating System Design: Locks and Condition Variables
Mar 05

Ram Kesavan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

Announcements

MP2 out; Due (3/23) 11:59 pm CT
MP1 20% students 1:1s with TA on-going
    Grades finalized by next week

Mid-term online survey: next week
Midterms: **Mar 26th** (Piazza announcement)
    Last Name starts with A-L: DCL 1310
    Last Name starts with M-Z: Everitt Lab 2310

Mar 12 lecture: handled by TAs; I'm traveling

AGENDA / LEARNING OUTCOMES

Last Lecture:

- Motivation for Concurrency
- Need for synchronization primitives
- Critical sections & other models

Today:

- Locks: how to use and how to implement
- HW atomic instructions to implement locks
- What do do when waiting for a lock
- If we have time, condition variables

Synchronization

Generic synchronization: for all models from previous slides

Goal: Learn how synchronization primitives are built (lib + OS)
With a little help from our friend…the hardware

Monitors
Locks
Semaphores
Condition Variables

Atomic Load
XCGH, Test&Set, etc.
Atomic Store
Disable Interrupts

LOCKS

Locks

Goal: Provide mutual exclusion (mutex)

Allocate and Initialize

- pthread_mutex_t mylock = PTHREAD_MUTEX_INITIALIZER;

Acquire

- Acquire exclusive access to lock
- Wait if lock is not available; some other thread has it
- Spin or block (relinquish CPU) while waiting
- pthread_mutex_lock(&mylock);

Release

- Release exclusive access to lock; let another thread at it
- pthread_mutex_unlock(&mylock);

LOCK IMPLEMENTATION GOALS

Goals:

- *Mutual exclusion*
  Only one thread in critical section at a time
- *Progress* (deadlock-free)
  If many simultaneous acquirers, must allow one to proceed
- *Bounded Waits* (starvation-free)
  Must eventually allow each waiting thread to enter

Fairness: Each acquirer waits for same amount of time (approx fifo)

Performance: CPU is not wasted unnecessarily

IMPLEMENT LOCKS BY DISABLING INTERRUPTS

Turn off interrupts for critical sections
- Prevent timer interrupt, so no context switch possible
- Don't make any blocking system calls

Code between interrupts executes atomically

Can this work?

```
acquire(&lock);

// critical
//   section
//     code
//       here

release(&lock);
```

```
void acquire(lockT *l) {
    disableInterrupts();
}

void release(lockT *l) {
    enableInterrupts();
}
```

IMPLEMENT LOCKS w/ Load+Store

Use a single **shared** lock variable

```
// shared variable
boolean lock = false;
void acquire(boolean *lock) {
    while (*lock); // spin…
    *lock = true;
}
void release(boolean *lock) {
    *lock = false;
}
```

Does this work?

Implementing Synchronization

Disabling Interrupts

- Used by OS in very special circumstances
  - Eg. within an interrupt handler

Locking using loads/stores (Peterson's algorithm)

- Correct but it suffers from wasteful spin-waiting

But locks today are built using support from hardware (and OS)

XCHG: Atomic Exchange

Atomic instructions that allow a test+set atomically: XCHG, CAS

```
// pseudo-code for xchg (load + store in a single instruction!)
// return what was pointed to by addr
// also, store newval into addr
int xchg(int *addr, int newval) {
        int old = *addr;
        *addr = newval;
        return old;
}
```

CAS

Compare: check value, if success then Swap: with new value

LOCK Implementation with XCHG

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    lock->flag = 0;
}

// Use the XCHG instruction
void acquire(lock_t *lock) {
    ????;
    // spin-wait (do nothing)
}

void release(lock_t *lock) {
    lock->flag = ??;
}
```

int xchg(int *addr, int newval)

Using Atomic Instructions

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    // 0: lock is available, 1: lock is held
    lock->flag = 0;
}

void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        ; // spin-wait (do nothing)
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Test – return old value

Set – set the passed in value

HW does them atomically!

Other Atomic HW Instructions

```
int CompareAndSwap(int *addr, int expected, int new) {
    int actual = *addr;
    if (actual == expected)
        *addr = new;
    return actual;      // OR return true if comparison succeeded
}
```

```
// use the CAS instruction
void acquire(lock_t *lock) {
    while (CompareAndSwap(&lock->flag,  ,  ) ==  ) ;
    // spin-wait (do nothing)
}
```

LOCK IMPLEMENTATION GOALS

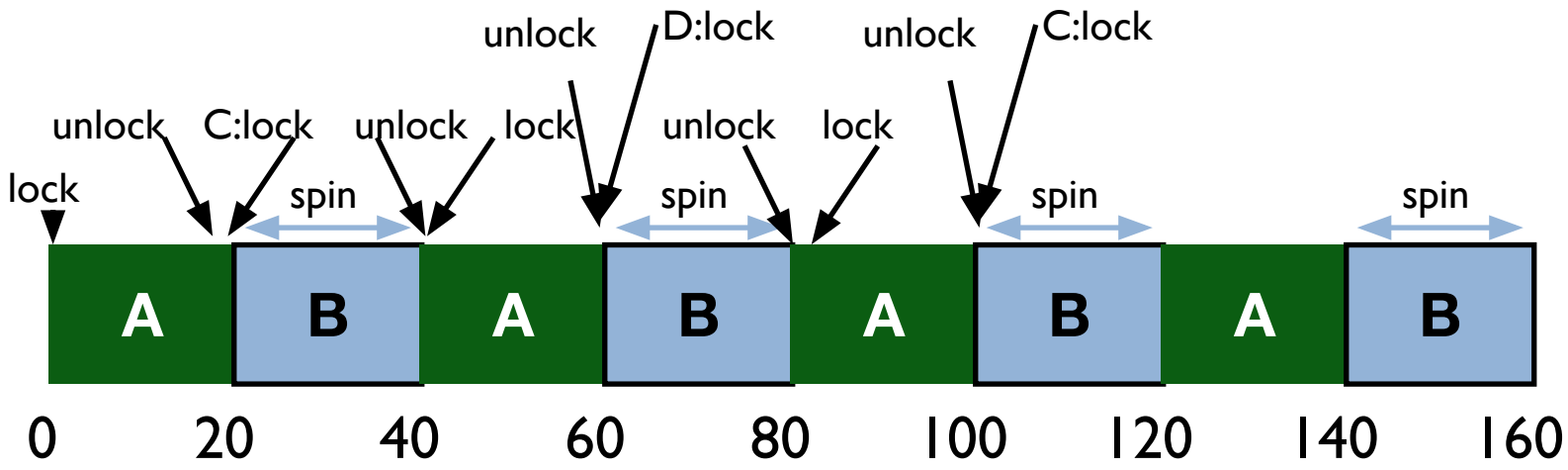
Correctness

- *Mutual exclusion (safety)*
  Only one thread in critical section at a time
- *Progress* (liveness)
  If several simultaneous requests, must allow one to proceed

Fairness: Does each thread have a fair shot at acquiring?
Does anybody starve?

Performance: Does CPU get wasted?

BASIC SPINLOCKS ARE UNFAIR



Scheduler is totally unaware of locks/unlocks.

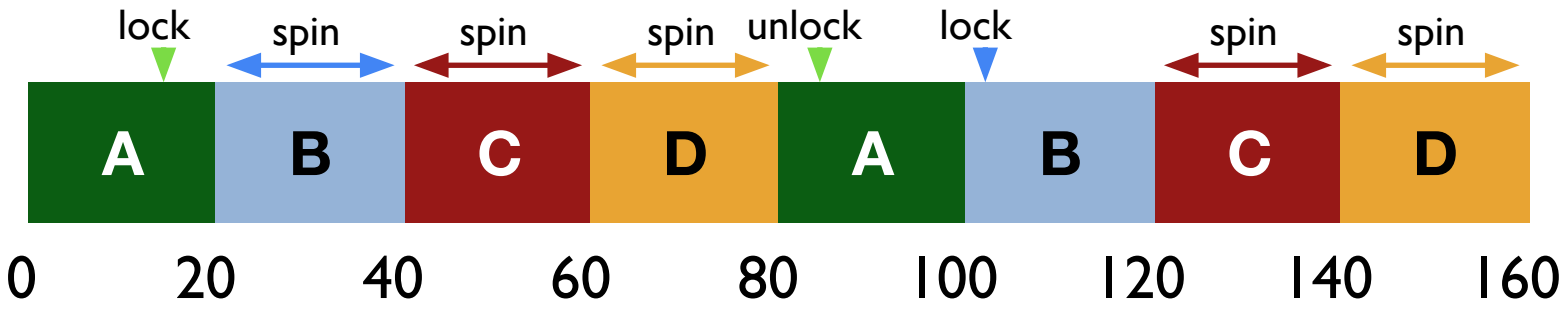
B is unlucky - not guaranteed to acquire lock

SPINLOCK PERFORMANCE

Slow because…

- locks may be held for a long time
    - Large critical sections
- Spinning is wasted CPU cycles

CPU SCHEDULER IS IGNORANT

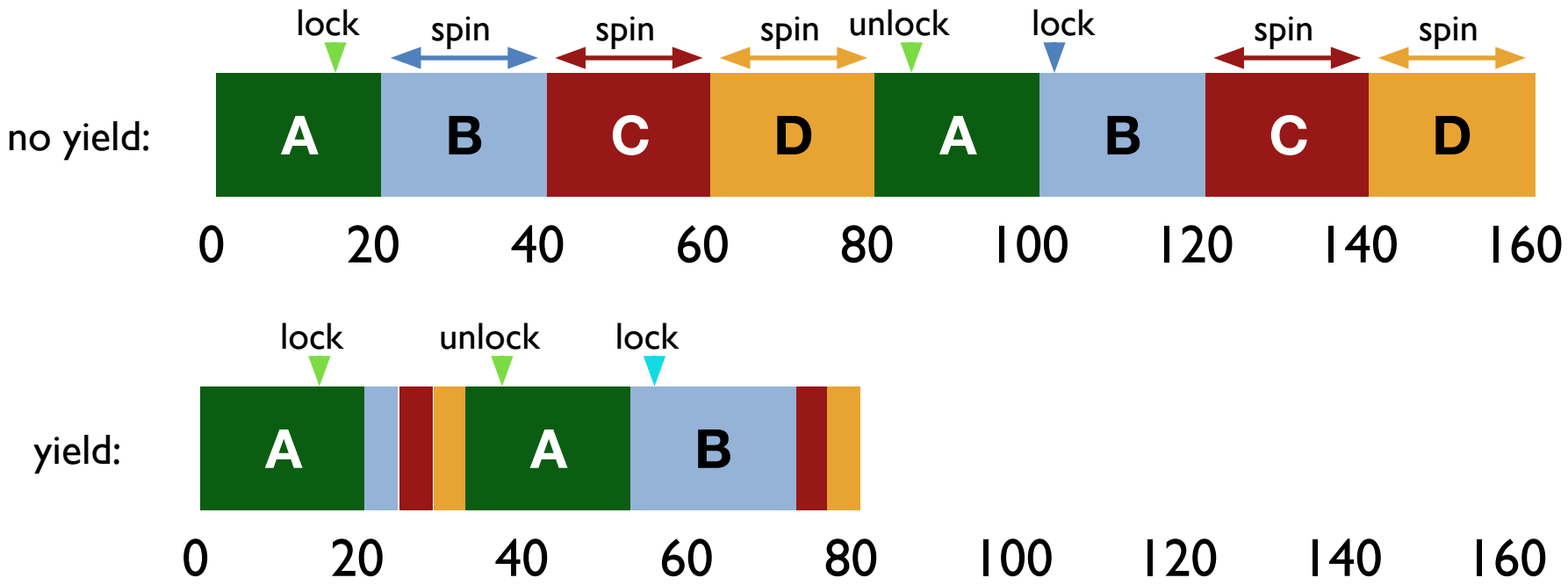


What if thread A is de-scheduled while in its critical section?

CPU scheduler may run **B, C, D** instead of **A**

even though **B, C, D** are waiting for **A**

Yield Instead of Spin



Lock Implementation: Block when Waiting

Remove waiting threads from scheduler runnable queue

Scheduler won't run any waiting thread

- CPU cycles saved

Support in Solaris OS: park(), unpark()

- park(): Solaris puts thread to SLEEPING state
- unpark(): Solaris transitions thread to READY state

Support in Linux: futex (fast userspace mutex)

Block when Waiting

```
typedef struct {
    bool lock = false;
    bool guard = false;
    queue_t q;
} LockT;
```

acquire(&lock);

// critical section

release(&lock);

```
void acquire(LockT *l) {
    while (XCHG(&l->guard, true));
    if (l->lock) {
            qadd(l->q, tid); // use gettid()
            l->guard = false;
            park();     // blocked
    } else {
            l->lock = true;
            l->guard = false;
    }
}

void release(LockT *l) {
    while (XCHG(&l->guard, true));
    if (qempty(l->q))
            l->lock=false;
    else
            unpark(qremove(l->q)); // wakeup
    l->guard = false;
}
```

Block when Waiting

(a) Why is **guard** used? What does it protect?

(b) Why okay to **spin** on guard?

(c) In release(), why not set lock=false when unpark?

```
void acquire(LockT *l) {
    while (XCHG(&l->guard, true));
    if (l->lock) {
            qadd(l->q, tid);
            l->guard = false;
            park();     // blocked
    } else {
            l->lock = true;
            l->guard = false;
    }
}

void release(LockT *l) {
    while (XCHG(&l->guard, true));
    if (qempty(l->q))
            l->lock=false;
    else
            unpark(qremove(l->q)); // wakeup
    l->guard = false;
}
```

Block when Waiting

(d) What if the order of guard=false and park() is flipped?

(e) Is there a case where a thread may indefinitely sleep?

```
void acquire(LockT *l) {
    while (XCHG(&l->guard, true));
    if (l->lock) {
            qadd(l->q, tid);
            l->guard = false;
            park();     // blocked
    } else {
            l->lock = true;
            l->guard = false;
    }
}

void release(LockT *l) {
    while (XCHG(&l->guard, true));
    if (qempty(l->q))
            l->lock=false;
    else
            unpark(qremove(l->q)); // wakeup
    l->guard = false;
}
```

FINAL VERSION

```
typedef struct {
    bool lock = false;
    bool guard = false;
    queue_t q;
} LockT;
```

```
void acquire(LockT *l) {
    while (XCHG(&l->guard, true));
    if (l->lock) {
            qadd(l->q, tid);
            setpark();     ←– NEW CODE
            l->guard = false;
            park();      // blocked
    } else {
            l->lock = true;
            l->guard = false;
    }
}

void release(LockT *l) {
    while (XCHG(&l->guard, true));
    if (qempty(l->q))
            l->lock=false;
    else
            unpark(qremove(l->q)); // wakeup
    l->guard = false;
}
```

setpark() fixes race condition:
park() won't block if unpark() occurred after setpark()

Spin-Waiting vs Blocking

Optimal approach depends on the circumstance

Uniprocessor

- Waiting thread should always relinquish processor
  - So that process (with lock) is scheduled
- Associate queue of waiters with each lock

Multiprocessor

- Thread holding lock may be running, may release lock soon
- Spin vs block: depends on how long (t) before lock is released
  - soon (small t) -> Spin
  - not so soon (large t)-> Block
  - “soon” is relative to context-switch cost, C

When to Spin-Wait vs Block?

How much CPU time is wasted when spin-waiting?
How much wasted when blocking?
What is the best action when $t < C$?
When $t > C$?
Problem: Requires knowledge of future

Hybrid approach: Two-phase Locks (not the distributed systems version!)

- Spin for some time ($< C$)
- Block after that

Synchronization

Generic synchronization: for all models from previous slides

Goal: Learn how synchronization primitives are built (lib + OS)

With a little help from our friend…the hardware

Monitors

Locks

Semaphores

Condition Variables

Atomic Load

XCGH, Test&Set, etc.

Atomic Store

Disable Interrupts

CONCURRENCY OBJECTIVES

Mutual exclusion (A and B don't both run critical section)

- solved with *locks*

Ordering (B runs only after A has done its thing)

- solved with *condition variables* and *semaphores*

ORDERING EXAMPLE: JOIN

```
pthread_t p1, p2;

pthread_create(&p1, NULL, mythread, "A");
pthread_create(&p2, NULL, mythread, "B");
// join waits for a pthread to exit
pthread_join(p1, NULL);
pthread_join(p2, NULL);

printf("main: children threads are done\n”);
return 0;
```

how to implement join()?

CONDITION VARIABLES

Condition Variable: queue of waiting threads

B waits for a signal on CV before running

- wait(CV, …)
- Don't spin while waiting

A sends signal to CV when time for ***B*** to run

- signal(CV, …)

CONDITION VARIABLES

wait(cond_t *cv, mutex_t *lock)

- the lock must be held before calling wait()
- put caller to sleep & release the lock (atomically)
- returns when awoken, with lock reacquired

signal(cond_t *cv)

- if (one or more threads are waiting)

    wake up one of them

- else

    return without doing anything

JOIN IMPLEMENTATION: ATTEMPT 1

Parent:

```
void thread_join() {
    mutex_lock(&m);     // P1
    cond_wait(&c, &m); // P2
    mutex_unlock(&m);  // P3
}
```

Child:

```
void thread_exit() {
    mutex_lock(&m);    // C1
    cond_signal(&c);   // C2
    mutex_unlock(&m); // C3
}
```

Example schedule:

Parent:	P1	P2				P3
Child:			C1	C2	C3	

JOIN IMPLEMENTATION: ATTEMPT 1

Parent:

```
void thread_join() {
    mutex_lock(&m);     // P1
    cond_wait(&c, &m); // P2
    mutex_unlock(&m);  // P3
}
```

Child:

```
void thread_exit() {
    mutex_lock(&m);     // C1
    cond_signal(&c);    // C2
    mutex_unlock(&m);  // C3
}
```

Example broken schedule:

RULE OF THUMB 1

Keep state in addition to CV's!

CV's are used to signal threads when that state changes

If state is already as needed, thread doesn't wait for a signal!

JOIN IMPLEMENTATION: ATTEMPT 2

Parent:

```
void thread_join() {
    mutex_lock(&m);          // P1
    if (done == 0)           // P2
        cond_wait(&c, &m);// P3
    mutex_unlock(&m);        // P4
}
```

Child:

```
void thread_exit() {
    done = 1;          // C1
    cond_signal(&c); // C2
}
```

Fixes previous broken schedule

Parent:		P1	P2	P3	P4
Child:	C1	C2			

JOIN IMPLEMENTATION: ATTEMPT 2

Parent:

```
void thread_join() {
    mutex_lock(&m);          // P1
    if (done == 0)           // P2
        cond_wait(&c, &m);// P3
    mutex_unlock(&m);        // P4
}
```

Child:

```
void thread_exit() {
    done = 1;          // C1
    cond_signal(&c); // C2
}
```

An example broken schedule:

JOIN IMPLEMENTATION: CORRECT

Parent:

```
void thread_join() {
    mutex_lock(&m);          // P1
    if (done == 0)           // P2
        cond_wait(&c, &m); // P3
    mutex_unlock(&m);        // P4
}
```

Child:

```
void thread_exit() {
    mutex_lock(&m);      // C1
    done = 1;            // C2
    cond_signal(&c);     // C3
    mutex_unlock(&m);    // C4
}
```

Parent: P1 P2 P3 P4

Child: C1 C2 C3 C4

Use mutex to ensure no race between interacting with state and wait/signal

CV RULE OF THUMB 2

1. Modify/check state with mutex held
2. Mutex is required to ensure state doesn't change between checking the state and waiting on CV
3. Always use a while instead of an if when checking cond

```
mutex_lock(&m);
while (done == 0) {
    cond_wait(&c, &m);
}
mutex_unlock(&m);
```

```
mutex_lock(&m);
if (done == 0) {
    cond_wait(&c, &m);
}
mutex_unlock(&m);
```

PRODUCER/CONSUMER PROBLEM

EXAMPLE: UNIX PIPES

Implementation:

- reads/writes to buffer require locking
- when buffers are full, writers must wait
- when buffers are empty, readers must wait

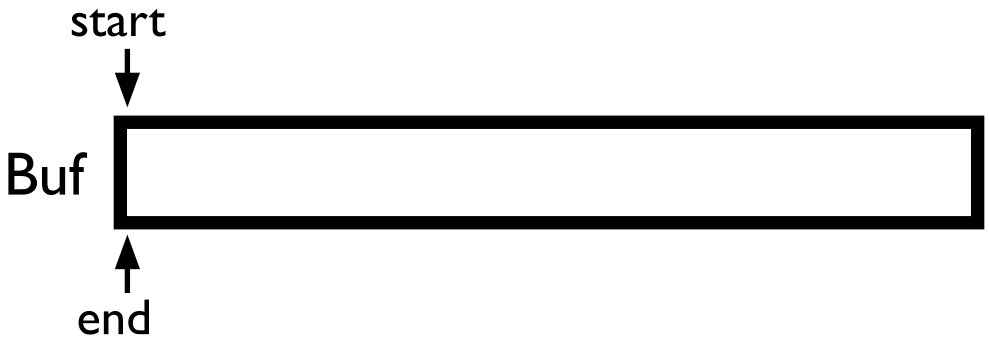
EXAMPLE: UNIX PIPES

A pipe may have many writers and readers

Internally, there is a finite-sized buffer

Writers add data to the buffer
- Writers have to wait if buffer is full

Readers remove data from the buffer
- Readers have to wait if buffer is empty



PRODUCER/CONSUMER PROBLEM

Producers generate data (like pipe writers)

Consumers grab data and process it (like pipe readers)

Producer/consumer problems are frequent in systems (e.g. web servers)

General strategy use condition variables to:

- make producers wait when buffers are full
- make consumers wait when there is nothing to consume

Producer/Consumer Example

Start with easy case:

- 1 producer thread
- 1 consumer thread
- 1 shared buffer to fill/consume (max = 1)

Numfull = number of slots currently filled

Numfull = 0 initially

Thread 1 state:

```
void *producer(void *arg) {
    while(1) {
        mutex_lock(&m);
        if(numfull == max)
            cond_wait(&cond, &m);
        do_fill();
        cond_signal(&cond);
        mutex_unlock(&m);
    }
}
```

Thread 2 state:

```
void *consumer(void *arg) {
    while(1) {
        mutex_lock(&m);
        if(numfull == 0)
            cond_wait(&cond, &m);
        int tmp = do_get();
        cond_signal(&cond);
        mutex_unlock(&m);
        printf("%d\n", tmp);
    }
}
```

WHAT ABOUT 2 CONSUMERS?

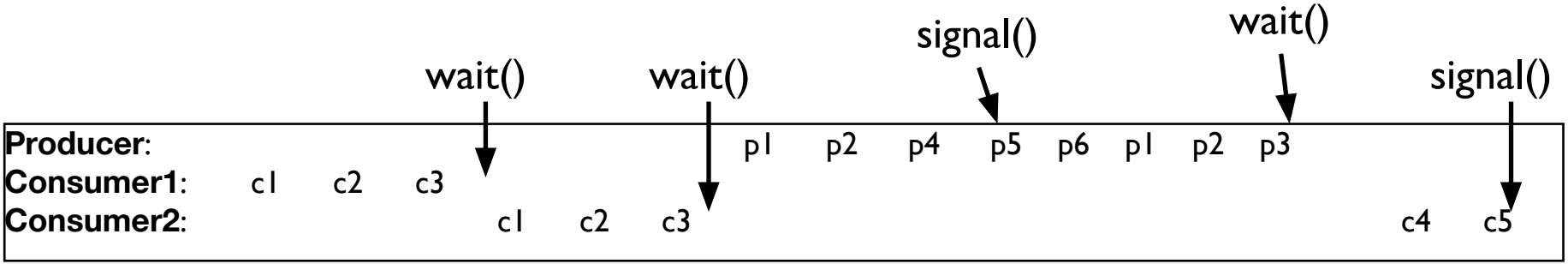
Can you find a problematic timeline with 2 consumers (still 1 producer)?

```
void *producer(void *arg) {
    while(1) {
        Mutex_lock(&m); // p1
        if(numfull == max) //p2
            Cond_wait(&cond, &m); //p3
        do_fill(); // p4
        Cond_signal(&cond); //p5
        Mutex_unlock(&m); //p6
    }
}
```

```
void *consumer(void *arg) {
    while(1) {
        Mutex_lock(&m);  // c1
        if(numfull == 0) // c2
            Cond_wait(&cond, &m); // c3
        int tmp = do_get(); // c4
        Cond_signal(&cond); // c5
        Mutex_unlock(&m); // c6
        printf("%d\n", tmp); // c7
    }
}
```

```
void *producer(void *arg) {
    while(1) {
        Mutex_lock(&m); // p1
        if(numfull == max) //p2
            Cond_wait(&cond, &m); //p3
        do_fill(); // p4
        Cond_signal(&cond); //p5
        Mutex_unlock(&m); //p6
    }
}
```

```
void *consumer(void *arg) {
    while(1) {
        Mutex_lock(&m);  // c1
        if(numfull == 0) // c2
            Cond_wait(&cond, &m); // c3
        int tmp = do_get(); // c4
        Cond_signal(&cond); // c5
        Mutex_unlock(&m); // c6
        printf("%d\n", tmp); // c7
    }
}
```



HOW TO WAKE THE RIGHT THREAD?

Wake all the threads!? (Broadcast)

Better solution (usually): use two condition variables

Producer/Consumer: Two CVs

```
void *producer(void *arg) {
    for (int i = 0; i < loops; i++) {
        Mutex_lock(&m); // p1
        if (numfull == max) // p2
            Cond_wait(&empty, &m); // p3
        do_fill(i);  // p4
        Cond_signal(&fill); // p5
        Mutex_unlock(&m); //p6
    }
}
```

```
void *consumer(void *arg) {
    while (1) {
        Mutex_lock(&m);
        if (numfull == 0)
            Cond_wait(&fill, &m);
        int tmp = do_get();
        Cond_signal(&empty);
        Mutex_unlock(&m);
    }
}
```

Solves the previous problem…
But can you find a bad schedule?

Producer/Consumer: Two CVs

```
void *producer(void *arg) {
    while(1) {
        Mutex_lock(&m); // p1
        if (numfull == max) // p2
            Cond_wait(&empty, &m); // p3
        do_fill();  // p4
        Cond_signal(&fill); // p5
        Mutex_unlock(&m); //p6
    }
}
```

```
void *consumer(void *arg) {
    while (1) {
        Mutex_lock(&m);
        if (numfull == 0)
            Cond_wait(&fill, &m);
        int tmp = do_get();
        Cond_signal(&empty);
        Mutex_unlock(&m);
    }
}
```

1. consumer1 waits because numfull == 0
2. producer increments numfull, wakes consumer1
3. before consumer1 runs, consumer2 runs, grabs entry, sets numfull=0.
4. consumer1 then reads bad data.

Producer/Consumer: Two CVs and WHILE

```
void *producer(void *arg) {
    for (int i = 0; i < loops; i++) {
        Mutex_lock(&m); // p1
        while (numfull == max) // p2
            Cond_wait(&empty, &m); // p3
        do_fill(i);  // p4
        Cond_signal(&fill); // p5
        Mutex_unlock(&m); //p6
    }
}
```

```
void *consumer(void *arg) {
    while (1) {
        Mutex_lock(&m);
        while (numfull == 0)
            Cond_wait(&fill, &m);
        int tmp = do_get();
        Cond_signal(&empty);
        Mutex_unlock(&m);
    }
}
```

No concurrent access to shared state

Every time lock is acquired, assumptions are reevaluated

A consumer will get to run after every do_fill()

A producer will get to run after every do_get()

GOOD RULE OF THUMB 3

Whenever a lock is acquired, recheck assumptions about state!

Another thread could grab lock in between signal and wakeup from wait

Note that some libraries also have “spurious wakeups” (may wake multiple waiting threads at signal or at any time)

Good stress test: change your signal to broadcast and see if your code still works

HOARE VS MESA SEMANTICS

- Mesa (used widely)
  - Signal puts waiter on ready list
  - Signaler keeps lock and processor
  - Not necessarily the waiter runs next
- Hoare (almost no one uses)
  - Signal gives processor and lock to waiter
  - Waiter runs when woken up by signaler
  - When waiter finishes, processor/lock given back to signaler

SUMMARY: RULES OF THUMB FOR CVS

1. Keep state in addition to CV's

2. Always wait or signal with lock held

3. Always recheck state when thread wakes up from a wait

   Use a while instead of an if